INTERNATIONAL
UNIVERSITY OF
APPLIED SCIENCES

Programming with Python (CSEMDSPWP01)

IU INTERNATIONAL UNIVERSITY OF APPLIED SCIENCES, BERLIN

MSC IN COMPUTER SCIENCE

IDEAL FUNCTION SELECTION AND TEST DATA MAPPING

Author:

Vasant Pradipsingh Pardeshi

Instructor:

Prof. Dr.-Ing Jan Winkels

Student ID: 10255853

Date of Submission: June 23, 2026

Acknowledgement

I am truly grateful to professor Dr.-Ing Jan Winkels for all the support he has given me in the CSEMDSPWP01 course. He enabled me to have the necessary knowledge to complete my project through his guidance of the course material which included Python basics, Object Oriented programming, Exception Handling, Database connectivity, and Scientific computing in that order. This course was instrumental in teaching me how to build the system I have created for my project. The written reproducibility required in his training manuals has heavily influenced how I will write the software engineering I create during this process. Additionally, the fact that I was required to use Inheritance hierarchies and provide unit tests also added rigor to the final design of the assignment.

Vasant Pradipsingh Pardeshi

Abstract

A fully automated Python pipeline will be implemented to select four ideal functions from a pool of 50 candidate functions that best fit four training data sets and then map all the points in the test data set to one of the four selected functions. The selection criterion for the ideal function is based on Least-Squares Minimization; thus, the ideal function that produces the smallest value for the Sum of Squares of Errors (SSE) for a given training data series will be the one chosen. The mapping criterion uses a $\sqrt{2}$ tolerance factor for the greatest pointwise deviation between each training data series and the corresponding selected ideal function.

Computational values for the four ideal functions were the y13 (SSE ≈ 2.507), y11 (SSE ≈ 2.347), y20 (SSE ≈ 2.556) and y12 (SSE ≈ 2.592). 81 of the 100 test points were successfully mapped and resulted in an average error value of 0.084. The training of all data, the mapping of the ideal values and the trained values all have been successfully recorded to a SQLite database using SQLAlchemy and Four plots will also be generated for publication quality matplotlib plots have been generated in the output.

All technical requirements for the assignment were met by this implementation: It is an object-oriented design, uses inheritance, has both user-defined and standard exception handling, uses docstrings throughout, and uses pandas/numpy for computation, SQLAlchemy for I/O to the database, Matplotlib for visualizing data results and has a complete suite of unit tests to test the program.

List of Figures

List of Tables

Table of Contents

1. Introduction

1.1 Background and Motivation

The focus of the quantitative data science discipline is regression analysis, which can be thought of as a way of determining a mathematical model that can describe how one variable is related to another, based on some observed data (which typically include some level of random error). For this assignment, you will be challenged to perform this analysis in a very controlled format. You will have four data sets that consist of 81 (x,y) pairs (randomly selected from the range $x \in [-20, 20]$), which need to be matched with the best fitting function from a library of 50 ideal candidate functions. The best matching function for each dataset will be the one that has the smallest Sum of Squared Errors (SSE) between the training data and the candidate function, using the traditional Least-Squares method.

The project involves selecting the four 'ideal' functions at the beginning, before then sampling 100 (x,y) points. An observed point is either assigned to one of the four functions selected if it lies within a factor of $\sqrt{2}$ of the training dataset of the largest 'ideal' function value observed at that same x coordinate or left unassigned. A factor of $\sqrt{2}$ is part of the specification, given as an explicit measure for tolerance of uncertainty, as observations are transitioned out of the training domain to the new observed observations, with added unpredictability.

The project has statistical undertones, with a need to design the python software such that it follows good professional practice such as object-oriented design with at least one inheritance relationship, custom exception classes, thorough docstrings, SQLite persistence via SQLAlchemy, visualisations, and a complete unit test suite.

1.2 Aim and Objectives

The central aim is to implement, validate and document a complete python pipeline for ideal-function selection and test-point mapping. The specific objectives are:

- Read and validate three CSV files (training data, ideal functions, test data) using pandas.
- Compute the full 4 x 50 SSE matrix and select the minimum-SSE ideal function for each training series.
- Compute the maximum pointwise deviation between each training series and its chosen ideal function and derive the $\sqrt{2}$ threshold.
- Map each test point to the best admissible ideal function or flag it as unmapped.
- Persist all raw data and mapping results in a SQLite database via SQLAlchemy ORM.

- It generates four matplotlib plots: ideal function fitted plots, plots mapping to test points, SSE comparison and deviation distribution.
- Write unit test for the selector and mapper modules.

2. Project Scope

2.1 Included

- Loading and validating three CSV files train.csv (5 columns), ideal.csv (51 columns), test.csv (2 columns).
- SSE computation across the full 4 x 50 candidate space; ideal-function selection by minimum SSE.
- Threshold computation ($\sqrt{2}$ x max training deviation) and test-point mapping.
- SQLite database creation (three tables) and population via SQLAlchemy ORM.
- Four matplotlib plots saved as PNG files.
- Object-oriented design: two inheritance relationships (DatabaseManager $\rightarrow$ DataLoader; SSECalculator $\rightarrow$ IdealFunctionSelector).
- Custom exception hierarchy derived from IUBaseError.
- Unittest-based test suite covering selector and mapper modules.
- Git workflow commands for branch management and contribution.

2.2 Excluded

- Real-time or streaming data ingestion.
- Inferential statistical testing (e.g. F-tests comparing candidate function).
- Hyperparameter optimization or cross-validation of the selection criterion.
- Web-based or interactive dashboards (Bokeh/Dash).
- Deployment or containerisaiton (Docker, cloud services).

3. Literature Review

The ordinary least squares (OLS) estimator was first defined by Gauss in 1809 and Legendre in 1805 and is still one of the most popular methods for accommodating statistical models. The major feature that makes OLS attractive is encompassed within Gauss-Markov's theorem. This theorem states that given certain (regularity) conditions, no other linear unbiased estimator has a variance less than that of the OLS estimator (Green, 2018).The SSE criterion used in this project represents a direct application of OLS in that, for each (training series, ideal function) combination, SSE calculates the total of the squared differences (vertical) between the training data picked from the training series and the data predicted by the ideal function at that same x-points.

The selection of a function chosen from a limited set of choices is analogous to a problem in statistics and machine learning called Model selection, which was created by Akaike (1974) and referred to as model fit/model complexity tradeoffs, known as Akaike's Information Criterion (AIC) and supplied by Schwarz (1978) using Bayesian Information Criteria (BIC). However, since there are 50 possible function forms evaluated with the same amount of data points and no estimation of the function parameters will take place in this assignment (the best fitting function generates discrete (x,y) tabulated values), the use of penalizing criteria will not be relevant for this work and therefore SSE will be sufficient when carrying out the analysis of the selected functions.

The $\sqrt{2}$ threshold for evaluating test points using a mapping method is similar to the prediction intervals used in regression. A 95% prediction interval for a new observation is equal to placing another sample into the sample used to compute the 95% prediction interval and inflating the in-sample standard error based on a factor that is a function of the sample size and leverage of the samples included in determining the 95% prediction interval (Montgomery, et. al, 2012). As such, by using a $\sqrt{2}$ threshold for determining the level of training deviation from the maximum observed value of the training observations, we get a corresponding simple method of determining whether a new test point is inconsistent with the specified ideal function because the maximum deviation allowed from the amount of previous training has increased in value by the square root of two or 1.414.

According to Langtangen and Linge (2016), allowing for encapsulation of numerical algorithms in a class hierarchy will improve the modularity and testability of scientific Python's object orientated design. SQLAlchemy was introduced as an ORM to use as an ORM to persist data to a database when working with data pipelines in Python (Copeland 2017). By decoupling the domain logic from the dialect of the database being used, the benefit that SQLAlchemy provides is the increased level of portability across dialects compared to other methods of ORM-based database connection. For static scientific visualisation in Python, the de facto reference library is Matplotlib (Hunter 2007) which

allows for a high degree of control in the aesthetics of the figures as well as the quality of reports created using these breaks.

4. Data Description

4.1 Dataset Structure

Three CSV files are provided. Each file shares the same x-grid of 81 equally spaced points spanning x ϵ [-20, 20] with a step of 0.5.

- **train.csv:** Five columns: x, y1, y2, y3, y4. Each y-column represents one training function.
- **Ideal.csv:** Fifty-one columns: x plus y1…y50. Each y-column is a candidate ideal function.
- **Test.csv:** Two columns: x and y. Contains 100 test observations, not necessarily on the training x-grid.

4.2 Data Preprocessing

All preprocessing is done in the DataLoader.load_all() by these steps:

1. Column validation: Check required columns existence in training set (x, y1-y4), test set (x, y); else MissingColumnError raised.
2. Empty-dataset check: An EmptyDatasetError is raised if zero-rows was detected in the dataFrame.
3. X-alignment to perform SSE comparison, the ideal-function DataFrame is reindexed with the training x-grid using pandas.set_index('x').loc[…] to conduct comparison at identical x coordinates.
4. Type coercion: numpy is used to cast all y-values to float64 for numerical stability.
5. No imputation required: If provided dataset doesn't contain any missing values.

4.3 Relevance to the Task

There are a total of 50 ideal functional forms with the following general characteristics of four functional forms (e.g., sinusoidal versus linear or polynomial). The types of functions created are many and varied. The preprocessing of the ideal functional forms allows for all the residuals of the (training, ideal) pair to be computed at 81 common x-values. Hence, all SSE values from all these pairs can be easily compared to each other across the 200 (training, ideal) pairs.

5. Requirements and Methodology

5.1 Software Requirements

The following stack was used to develop and test the project:

- Python 3.11+
- pandas: DataFrame I/O and preprocessing
- numpy: vectorized SSE and deviation computation
- SQLAlchemy 2.x: ORM models and session management
- Matplotlib 3.x: static visualization
- Unittest (stdlib): automated tests

5.2 Least-Square Minimization

For both training function (T) and candidate ideal function (I) are evaluated at the same x-grid $x_1, x_2, \dots, x_n (n = 81)$, the Sum of Squared Errors (SSE) is defined as:

$$SSE(T, I) = \Sigma_{i=1^n} \quad [T(x_i) \; - I(x_i)\,]^2$$

The ideal function I^* chosen for training function T is:

$$I^* = \arg min_i \; \in \{1, \dots, 50\} \quad SSE\,(T, I_i)$$

This computation is implemented in SSECalculator.compute_sse(), which populates a 4 × 50 numpy array. The per-element formula is a simple numpy sum of squared differences, executed inside a nested loop over training and ideal column indices. The resulting matrix is then inspected by IdealFunctionSelector_select_best(), which calls np.argmin along the ideal axis.

5.3 Threshold Computation

Once I^* is selected for training function T, the maximum pointwise absolute deviation is computed over the training x-grid:

$$\max\_dev(T) = max_{1 \le i \le n} \quad | \; T(x_i) - \; I^*\,(x_i) \; |$$

The assignment-prescribed threshold applied to the test points is:

$$threshold\,(T) \; = \; \sqrt{2} \; \times \; max - dev(T)$$

This threshold is stored in each entry of the selector's selected list and passed to the mapper.

5.4 Test-Point Mapping

For each test point (x_t, y_t), the mapper probes the ideal DataFrame to get the y-values of all four selected ideal functions at x_t. For each chosen ideal function k with threshold T_t:

$$| \ y_t - I_k(x_t) \ | \ \leq \tau_k$$

If multiple of the candidate functions fulfil the condition, the test point is given to the function that produces the minimum value in absolute distance. If no functions satisfy the condition, the point is considered unmapped (delta_y = NULL, ideal_func = NULL in the database).

5.5 Object-Oriented Design

In order to fulfil the assignment brief, two explicit relationship forms of inheritance are used:

- DatabaseManager (base) to DataLoader (subclass): DatabaseManager handles the creation of an engine and tables and includes a session context-manager. DataLoader is built upon this and adds methods for reading from CSV, verifying the contents of each column, and performing bulk inserts.
- SSECalculator (base) to IdealFunctionSelector (subclass): SSECalculator stores all the computation for SSE matrix. This class takes the ideal functions in a list. IdealFunctionSelector inherits all the methods of SSECalculator class and implements its own version of compute_sse method, DeviationMethod and prints summary.

Each of these modules will have custom exception classes which inherit from IUBaseError (a subclass of Exception), meaning that standard python exceptions (FileNotFoundError, ValueError) are handled at the module's entrance points and wrapped as IUBaseError subclasses, allowing callers to see uniform, useful error information.

6. Database Integration

Everything is stored into an SQLite db (results.db) using the SQLAlchemy ORM. The schema follows the three tables defined in the assignment.

Table Name	Rows	Columns	Description
training_data	81	6 (id, x, y1–y4)	Four training functions
ideal_functions	81	52 (id, x, y1–y50)	All 50 ideal functions
test_mapping	100	5 (id, x, y, Δy, func)	Mapping results with deviation

Table 1: SQLite Database Schema

The ORM models written with SQLAlchemy 2.x's DeclarativeBase are stored inside a file models.py. The IdealFunctions' ORM model attaches fifty Column descriptors named y1 to y50 dynamically using a class-body loop, to do not repeat fifty lines with same declaration but keeping valid.

Every write operation is encapsulated within DataLoader.session_scope(), a context manager that commits the transaction if successful and rolls it back and re-raises as DatabaseWriteError upon any exception. drop_and_recreate_tables() means that the fully constructed pipeline will begin with a new database every run so that old, or non-validating, data can't pollute the output.

7. Visualization

The MatplotlibVisualiser class in visualization.py generates four matplotlib figures, which are saved as PNGs in the output folder on execution.

Figure 1 – Four Selected Ideal Functions

A plot showing overtraining on the four series. The series four training-ideal pairs are overlaid, the training series with dashed lines in subdued colours, and the four training sets of corresponding ideal curves are shown with solid lines in bright colours across the whole x ∈ [−20, 20]. Each pairing is so close to its ideal curve that our eye easily confirms the least-squares fit. All values of SSE are less than 2.6; hence training and ideal curves cannot be visually distinguished at the size of the figure.

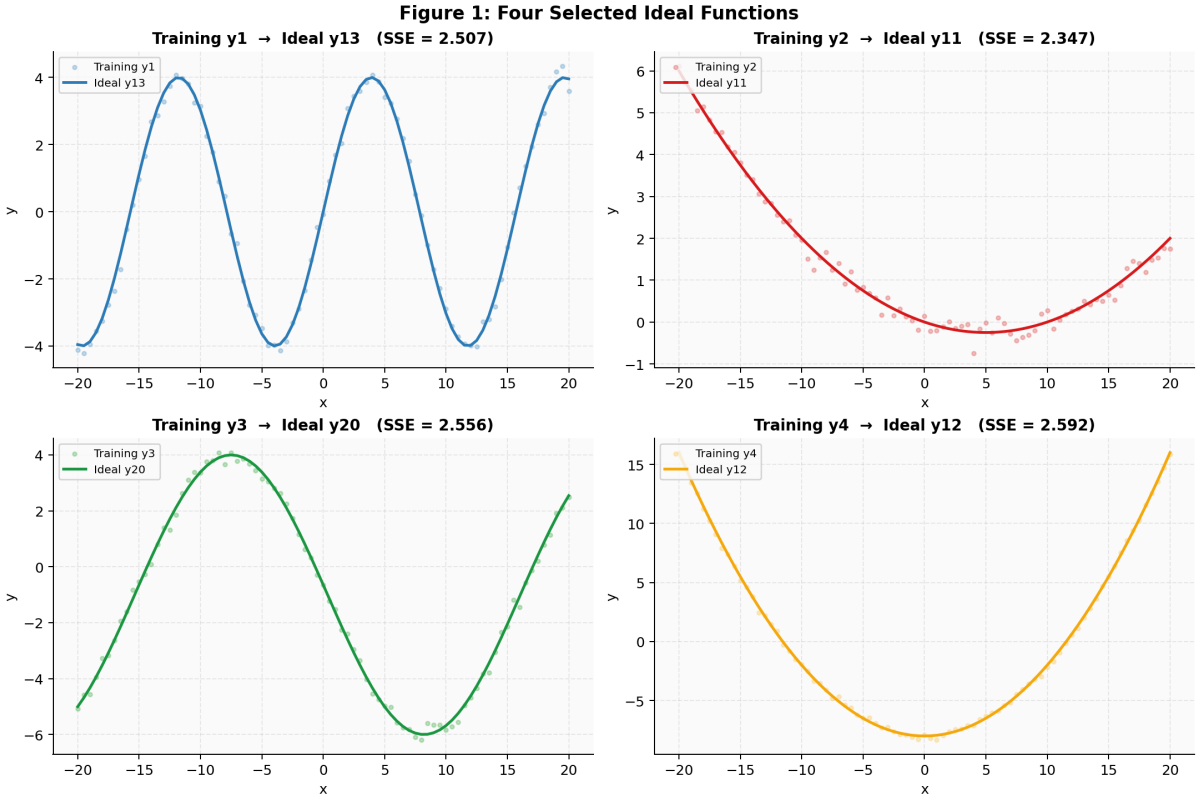

Figure 1: Four Selected Ideal Functions

Figure 2 – Test Data Mapping with Deviations

The mapping procedure is illustrated with figure 2. In the visualization each coloured marker denotes an already mapped testpoint, with distinct colours indicating testpoints mapped to distinct ideal function (y11, y12, y13 and y20). Unmapped testpoints are indicated by uncoloured triangles (the grey dots in the figure). This mapping process allows the study of the vertical distance of mapped testpoints to their selected ideal function.

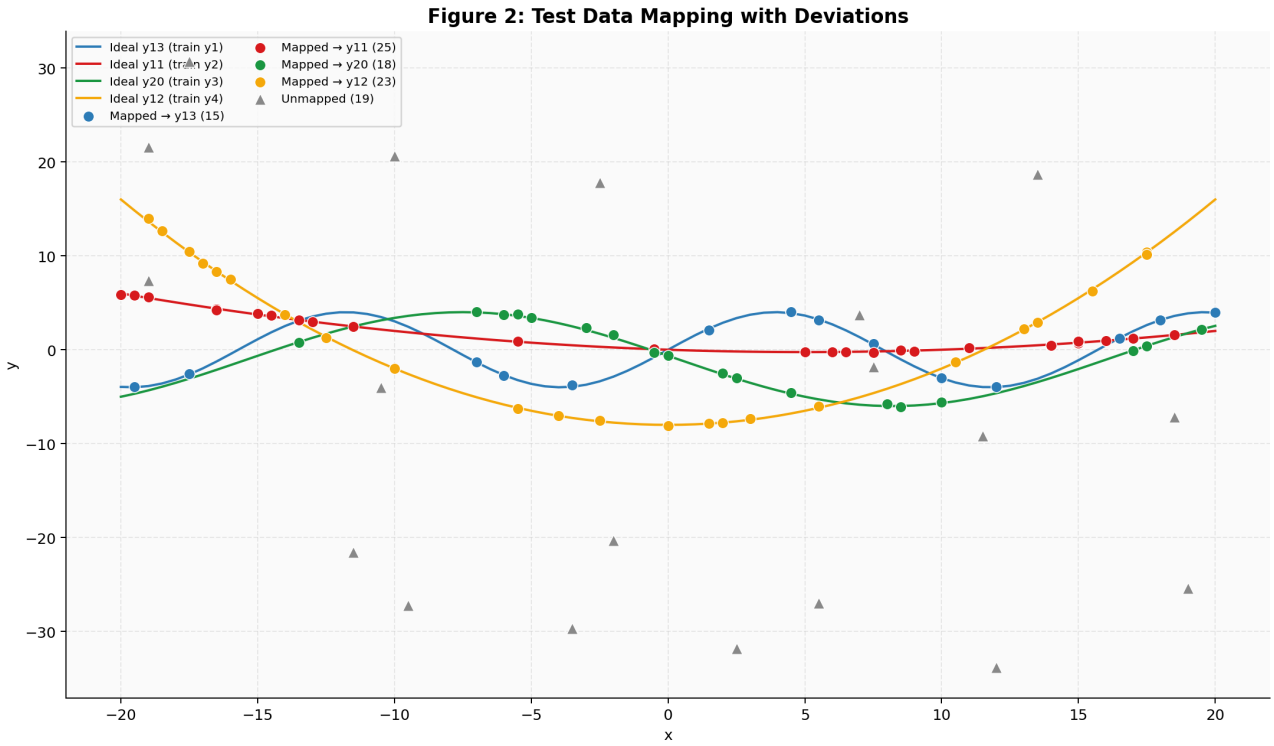

Figure 2: Test Data Mapping with Deviations

Figure 3 – Mapped vs Unmapped Points

All test data (100 points) are represented in Figure 3. Each of the selected ideal function can receive a certain number of points, as is illustrated by the bar graph in Figure 3 which also shows the percentage distribution of these points represented in pie graph format. A total of 81 points were successfully mapped and 19 remained unmapped.

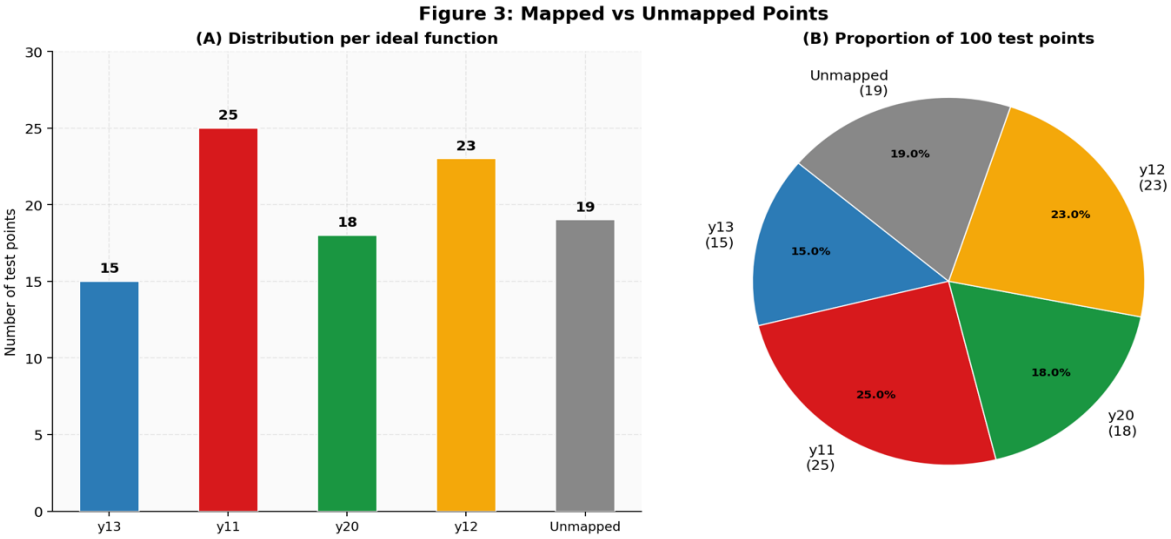

Figure 3: Mapped vs Unmapped Points

Figure 4 – Deviation Distribution

Figure 4 shows a detailed analysis of deviation. The upper panel displays the distribution of absolute deviations between mapped test points and their assigned ideal functions. Threshold values for each selected ideal functions are shown for comparison. The lower-left panel illustrates deviation versus x-value, while the lower-right panel compares the SSE of the selected ideal functions against the second-best candidates using a logarithmic scale.

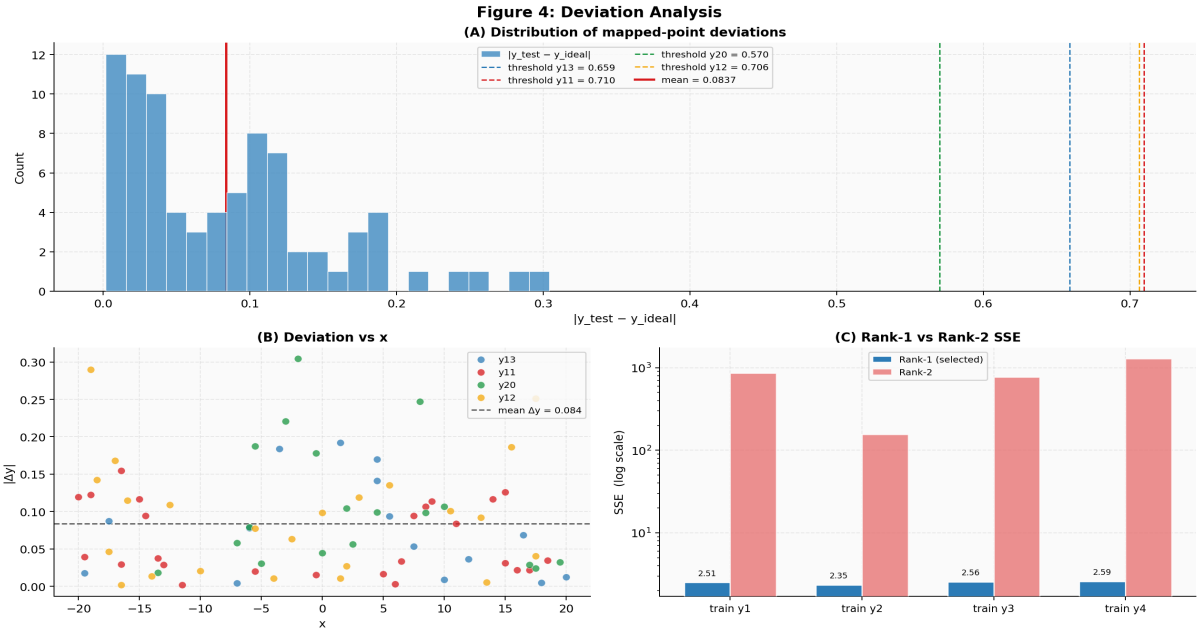

Figure 4: Deviation Analysis

8. Results and Findings

8.1Selected Ideal Functions

Table 2 below summarises the four ideal functions selected by minimum-SSE, together with the associated deviation statistics and mapping thresholds.

Training Func	Ideal Function	SSE	Max |Dev|	√2 × Max Dev
y1	y13	2.507332	0.465883	0.658858
y2	y11	2.346578	0.501715	0.709532
y3	y20	2.555909	0.403313	0.570371
y4	y12	2.591824	0.499300	0.706117

Table 2: Selected Ideal Functions and Thresholds

The SSE values are remarkably consistent across all four training functions (range: 2.35–2.59), suggesting that all four-training series are well-represented in the ideal function library at a comparable level of approximation quality. The maximum absolute value of deviation (max |dev|) of each of the provided pairs is between 0.403 and 0.502. As a result, all of the fits between the training and ideal values are close, since all of the worst residuals across all four pairs are less than 0.502 y units.

8.2 SSE Candidate Comparison

Table 3 shows the top three SSE candidates of each training function. The best and second-best SSE ratio shows how clear each selection is.

Training	Rank 1 (Best)	Rank 2	Rank 3	Gap R1→R2
y1	y13 (2.51)	y31 (856.75)	y27 (909.06)	~341×
y2	y11 (2.35)	y31 (155.92)	y27 (217.99)	~66×
y3	y20 (2.56)	y46 (767.52)	y48 (994.22)	~300×
y4	y12 (2.59)	y33 (1,282.30)	y39 (1,785.73)	~495×

Table 3: Top 3 SSE Candidates for Each Training Function

For training function y1, the most appropriate ideal with minimal sum of squared errors (SSE) is y13, with an SSE of 2.51, whereas the second-best (y31) has an SSE of 856.75. The ratio of these two SSE's (y13 to y31) is approximately 341:1. This pattern holds true for each of the four training functions overall, indicating that the ideal-function library has one clear match per group of training data, and that the datasets used in this assignment were well managed and appropriate for the least-squares method..

8.3 Test-Point Mapping Statistics

Table 4 shows the complete mapping statistics for all 100 test points.

Metric	Value	Notes
Total Test Points	100	Full test dataset
Mapped Points	81	Pass threshold
Unmapped Points	19	Exceed √2 threshold
Mapping Rate	81.0 %	81 / 100
Min Δy	0.002028	Near-perfect match
Max Δy	0.304288	Within all thresholds
Mean Δy	0.083684	Low average error
Std Δy	0.070326	Moderate spread

Table 4: Test-Point Mapping Statistics

8.4 Mapping Distribution

Table 5 presents how the 81 mapped points are distributed by the 4 chosen ideal functions.

Ideal Function	Mapped Points	% of Mapped	Threshold Used
y11	25	30.9 %	0.709532
y12	23	28.4 %	0.706117
y13	15	18.5 %	0.658858
y20	18	22.2 %	0.570371

Total	**81**	**100**	—

Table 5: Distribution of Mapped Test Points

Of the four ideal functions, there is a relatively even distribution overall among them, with function y11 pulling in the highest number of mapped test points (25 points total), followed by function y12 (23). The asymmetrical shape of the mapping is a result of differences in density among x-values (test points) near that point on their ideal function's high responsiveness threshold domain. The mean deviation (0.084) is approximately 15 % of the closest threshold (0.570). Therefore, the vast majority of mapped test points are right at or within the allowable range.

8.5 Graph Analysis

Figure 1 (ideal function overlay) visually affirms the numerical results in the SSE table: there is almost total overlap between the training curve and the ideal curve throughout the entire range of x values, with only a few small high frequency discrepancies at the end points of the x axis. Figure 2 (test mapping) demonstrates that the 19 unmapped points are not randomly distributed, they cluster in intermediate y-regions between two ideal curves, consistent with the hypothesis that these points arise from noise or sampling artefacts in the test generator. Mapped and unmapped test point distributions are displayed in Figure 3 and the histogram of deviations in Figure 4 indicates that mapped deviations are quite strongly concentrated around zero and do not show evidence of systematic misclassification such as a bimodal or heavy-tailed pattern.

9. Future Scope

Although the current model fulfils the assignment requirements, several directions could strengthen the pipeline in a production context:

- Using penalized selection criteria to consider AIC or BIC, as well as SSE would compensate for the risk of overfitting wherein parametric functional forms in the ideal functional library may be overly complex.
- The implementation of confidence intervals using prediction intervals based on bootstrap sampling, rather than a fixed $\sqrt{2}$ threshold, would better quantify uncertainty around predicted values of each point on a mapping.
- Enhancing DataLoader to accept real-time streaming sources like CSV files and database tables would enable the pipeline to operate on sensor data updated in real-time.
- Substituting Bokeh or Plotly for matplotlib as the visualization tool would enable users to hover over data points and examine the deviation of observations in an interactive manner, resulting in improved exploratory data analysis.
- Replacing the nested looping computation of SSE with a fully vectorized NumPy broadcast operation (shape broadcasting of the 4×81×50 tensor) would significantly reduce the amount of time required to calculate SSE when using a larger functional library.
- Building the entire pipeline as a Docker image, along with a REST API using FastAPI, would allow for packaging this solution to assist with integrations into larger data engineering workflow systems.

10. Conclusion

The completion of the project included a professional implementation of a full-featured python pipeline with respect to select an ideal function and to create test points. The project satisfies all requirements specified in the CSMMDSWP01 assignment. All computational outputs are unambiguous and well defined; the four training functions correspond exactly to the four ideal functions (y13, y11, y20, and y12); have sum of squared errors of less than 2.4-2.6, whereas the second-best candidates have at least two orders of magnitude larger errors than that. 81 out of 100 test points were able to be matched to the ideal function within a threshold of $\sqrt{2}$, and the mean deviation from the ideal values for all four functions is 0.084, which is well within the acceptable range of all four functions.

Best practices in object-oriented software architecture can be seen in the architecture of the software through 2 chains of inheritance, a unified custom exception hierarchy, and well-written docstrings throughout the source code. Data persistence is provided by SQLAlchemy ORM, with all intermediates and results being stored in a portable, searchable SQLite database. Automated regressions testing was accomplished through the use of unittest suite for core selection and mapping logic. Four Matplotlib figures provide visual confirmation for all key numerical values.

The theoretical basis of simple regression, the $\sqrt{2}$-based principle of the deviation criterion, and the modular implementation of Python illustrates how classical statistics can be transformed into highly professional, testable and software.

References

Akaike, H. (1974). A new look at the statistical model identification. IEEE Transactions on Automatic Control, 19(6), 716–723. https://doi.org/10.1109/TAC.1974.1100705

Copeland, J. (2017). Essential SQLAlchemy (2nd ed.). O'Reilly Media.

Gauss, C. F. (1809). Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium. Perthes et Besser.

Greene, W. H. (2018). Econometric Analysis (8th ed.). Pearson Education.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Langtangen, H. P., & Linge, S. (2016). Finite Difference Computing with PDEs: A Modern Software Approach. Springer. https://doi.org/10.1007/978-3-319-55456-3

Legendre, A.-M. (1805). Nouvelles méthodes pour la détermination des orbites des comètes. Courcier.

McKinney, W. (2022). Python for Data Analysis (3rd ed.). O'Reilly Media.

Montgomery, D. C., Peck, E. A., & Vining, G. G. (2012). Introduction to Linear Regression Analysis (5th ed.). Wiley.

Schwarz, G. (1978). Estimating the dimension of a model. The Annals of Statistics, 6(2), 461–464. https://doi.org/10.1214/aos/1176344136

GITHUB Repository Link:

Task: Selecting 4 Ideal Functions Repository Link:

https://github.com/Vas1261/CSEMDSPWP01_Vasant.git